

Estructuras de Datos

TDA Cola

DEFINICION

* Abunda este concepto, en la vida cotidiana

- * Cuando vamos al cine, para comprar las entradas
- * Cuando estamos en el supermercado, en el banco, etc.

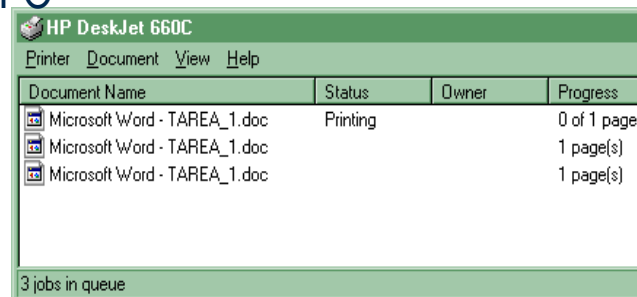
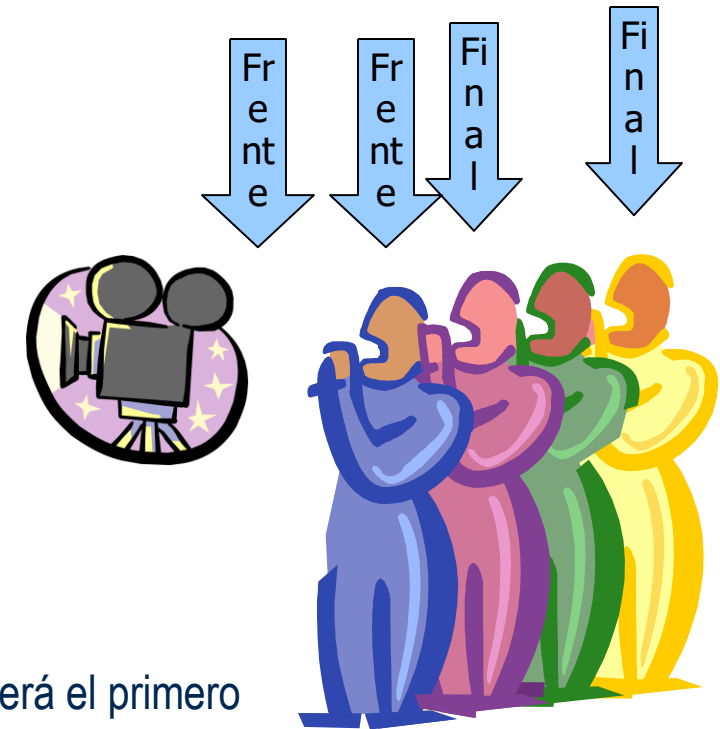
* Como funciona

- * Se puede decir que la cola tiene 2 extremos
 - * FRENTE, Y FINAL
- * Todo el que llega se ubica al final de la cola
- * Todo el que sale, lo hace por el frente de la cola
- * La cola es por turno
 - * El primero en llegar, tiene la seguridad de que será el primero en salir:

* FIRST IN FIRST OUT -> FIFO

* La computadora las utiliza:

- * Para manejar la impresión de documentos, tiempo compartido, etc.



Queue -> Cola
Cada documento que se manda a imprimir es "encolado", uno a uno es enviado a la impresora

OPERACIONES BASICAS

- * Al suponer que existe un TDA Cola, podemos:
 - * Cola Q;
- * Todo TDA presenta operaciones básicas, en este caso:
 - * **EnColar**
 - * *Insertar un elemento nuevo a la cola, al final de la misma,*
 - * *El final aumenta*
 - * **DesEnColar**
 - * *Cuando un elemento es removido de la cola*
 - * *Remueve el elemento del **frente***
 - * *Retorna el elemento removido*
 - * *No se puede ejecutar si la cola **EstaVacía***
- * Así como en la pila
 - * Cualquier intento de acceder a elementos en una Cola Vacía:
 - * **SUBDESBORDAMIENTO DE LA COLA**

COLAS: OPERACIONES

* Crear Cola

- * Crea una cola vacía

* Eliminar Cola

- * Elimina todos los elementos que en ella existen, para dejarla vacía

* Cola Esta Vacía

- * Determina si una cola esta vacía o no

* Cola Frente

- * Devuelve el elemento del frente de la cola

* Cola Final

- * Devuelve el elemento del final de la cola

* DesEnColar

- * Remueve el elemento del frente de la cola
- * Devuelve el elemento removido
- * No se puede remover si esta vacía

* EnColar

- * Inserta el elemento al final de la cola
- * El final aumenta en uno

IMPLEMENTACIONES

- * Las colas se pueden implementar de muchas formas
- * Una cola es
 - * Un conjunto ordenado de elementos
 - * Se supone que no tiene limite en su crecimiento
- * Una cola es una LISTA
- * Por lo tanto, la implementaciones posibles para la cola son
 - * Las mismas posibles para la LISTA
 - * Contigua(Estatica)
 - * Enlazada(Dinamica)
- * Sin embargo, en cada una de las posibles implementaciones
 - * Se pueden realizar ciertos cambios que se ajusten mas al concepto de la cola

RECORDEMOS ENTONCES..

- * Que para implementar la Cola
 - * Debo decidir sobre que implementación de lista basarme
- * Las implementaciones de las operaciones de la Cola
- * Solo llaman a las operaciones de las listas
 - * EnColar llamara a InsertarNodoInicio
 - * DesEnColar llamara a SacarNodoFinal
 - * ColaFrente llamara a ConsultarPrimero
 - * ColaFinal llamara a ConsultarUltimo
 - * Etc..

COLA: LISTA CONTIGUA

- * Si la lista contigua ya esta implementada
 - * Aun con los cambios que dijimos que íbamos a necesitar
- * La cola ya no necesita ser implementada
 - * Para implementar las operaciones de la cola
 - * Solo llamaremos a las operaciones de la lista ya implementadas

```
public bool estaLlena(){  
    return lista.estaLlena();  
}
```

```
public bool estaVacia(){  
    return lista.estaVacia();  
}
```

```
public bool encolar(Nodo nodo){  
    return lista.insertarNodoInicio(nodo);  
}
```

```
public Object colaFinal(){  
    return lista.consultarUltimo();  
}
```

```
public Object colaFrente(){  
    return lista.consultarPrimero();  
}
```

```
public Object desencolar(){  
    return lista.sacarNodoFinal();  
}
```

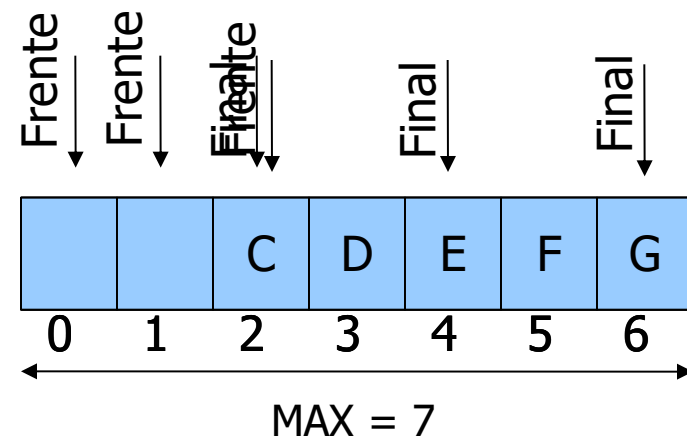
LSCONT: AJUSTANDOSE MEJOR A LA COLA

- * Dijimos, una cola tiene dos extremos
 - * Frente y Final
- * Siempre debemos llevar “control” de esas dos posiciones
- * La estructura que definiría el nuevo TDA
 - * Cuando la cola recién comienza,
 - * El Frente es el índice inicial del arreglo: 0
 - * El Final, como no hay, es -1
 - * La cola esta vacía
 - * Si $\text{Frente} > \text{Final}$
 - * La cola esta llena
 - * Si $\text{Final} == \text{MAX} - 1$
 - * A medida que se ingresan elementos
 - * El Final aumenta
 - * Cuando se eliminan
 - * El Frente, se mueve hacia delante

```
desencolar();  
encolar('F');  
encolar('G');
```

```
public class ColaEstatica{  
    Object cola[];  
    int Frente;  
    int Final;  
}
```

Ya no se pueden insertar mas elementos, aunque hay espacio atrás!



Cola Operaciones

```
public ColaLineal()
{
    frente = 0;
    fin = -1;
    listaCola = new TipoDeDato [MAXTAMQ];
}
```

```
public void insertar(TipoDeDato elemento) throws Exception
{
    if (!colaLlena())
    {
        listaCola[++fin] = elemento;
    }
    else
        throw new Exception("Overflow en la cola");
}
```

```
public TipoDeDato quitar() throws Exception
{
    if (!colaVacia())
    {
        return listaCola[frente++];
    }
    else
        throw new Exception("Cola vacia ");
}
```

```
public void borrarCola()
{
    frente = 0;
    fin = -1;
}

public TipoDeDato frenteCola() throws Exception
{
    if (!colaVacia())
    {
        return listaCola[frente];
    }
    else
        throw new Exception("Cola vacia ");
}
```

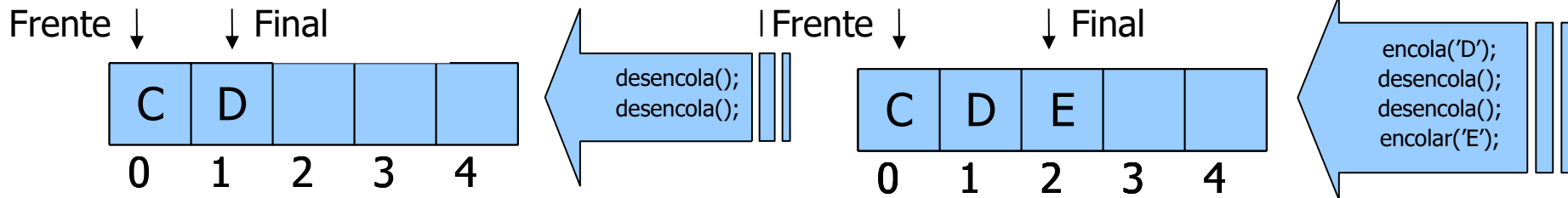
```
public boolean colaVacia()
{
    return frente > fin;
}

public boolean colaLlena()
{
    return fin == MAXTAMQ-1;
}
```

COMO RESOLVER EL PROBLEMA

- * Como vimos esta estrategia no es optima, se desperdicia espacio libre
- * El problema es al DesEnColar elementos
- * Usando arreglos lineales, hay dos posibles soluciones:

El Final
llego a
su
limite



* Retroceder con cada DesEncolamiento

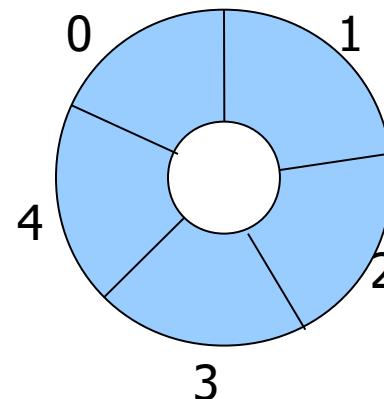
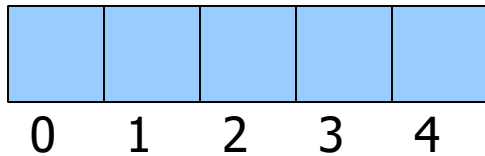
- * Al remover un elemento, no mover el **Frente** hacia delante
- * Mas bien, hacer que todos los elementos retrocedan una posición
- * Así no se desperdicia un “puesto” vacio
- * En esta estrategia el **Frente** nunca se mueve, Siempre es 0
- * El **Final** disminuye en una posición

* Cuando la Cola este Llena, Retroceder

- * Cuando **Final** llegue a su limite,
- * Retroceder todos los elementos hasta el inicio del arreglo
- * El **Frente**, que aumento durante el EnColamiento vuelve a ser 0
- * El **Final** disminuye el numero de “espacios vacios” que habian

ARREGLOS CIRCULARES

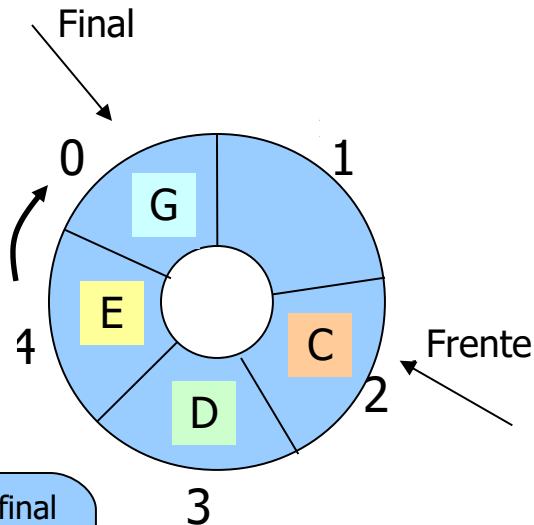
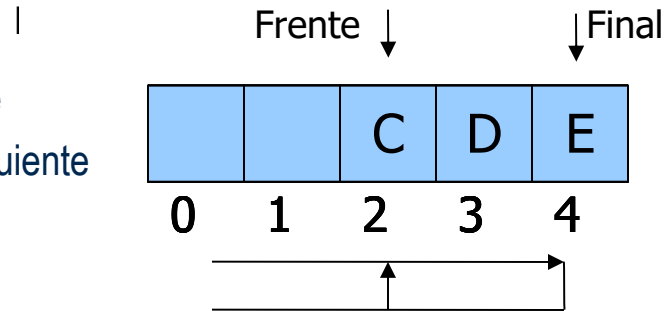
- * Sin embargo, ninguna de las anteriores es la ideal
- * Si una cola tiene 1000 elementos, y uno se DesEnCola
 - * Hay que mover 999 elementos
 - * Es muy poco eficiente
- * El problema del primer enfoque que revisamos era que
 - * Cuando el Final llegaba al limite, no se podía seguir avanzando y
 - * La Cola aparecía llena y esto no necesariamente era verdadero
- * Imaginemos un arreglo sin ese problema
 - * Al recorrer todo arreglo, se hace desde inicio 0, hasta fin MAX-1
 - * ¿Que tal si al recorrer el arreglo, al llegar a MAX-1, volvemos al primer elemento?
 - * Este seria un arreglo circular



¿Y COMO FUNCIONARIA?

* En un arreglo lineal, vimos:

- * Al EnColar, el Final, se mueve hacia el siguiente
- * Al DesEnColar, el Frente, se mueve hacia el siguiente



Al encolar el final
se mueve al
siguiente
Al desencolar el
frente se mueve al
siguiente

- * A medida que los elementos se desencolaron y encolaron
 - * La cola fue avanzando hacia la derecha
 - * El Final ha alcanzado el MAXimo permitido
 - * Sin embargo, al inicio del Arreglo, si hay espacios vacios
- * ¿Que hacer?
 - * Deseamos poder utilizar los elementos del inicio del arreglo
 - * Para colocar allí a los nuevos elementos
 - * Es decir, que una vez que el Final llegue a su limite
 - * El Final se pueda mover al inicio
 - * En un arreglo lineal, el siguiente del indice 4 es el indice 5
 - * Podemos "asumir" que el siguiente de 4, no es el 5, es 0

COLA Y COLA VACIA

Sacrificar un elemento
La cola estará llena no con
MAX elementos, sino con
MAX -1
Es cuando el elemento
siguiente del siguiente del
Final, es igual al Frente

* En un arreglo circular, se calcula:

* Si el índice $= \text{MAX} - 1$ entonces

* siguiente = 0

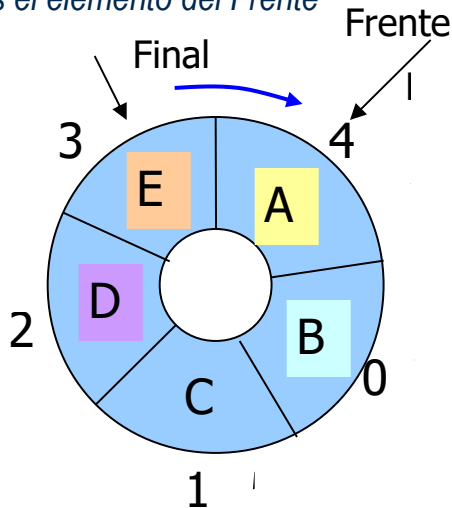
* Si no

* siguiente = índice + 1

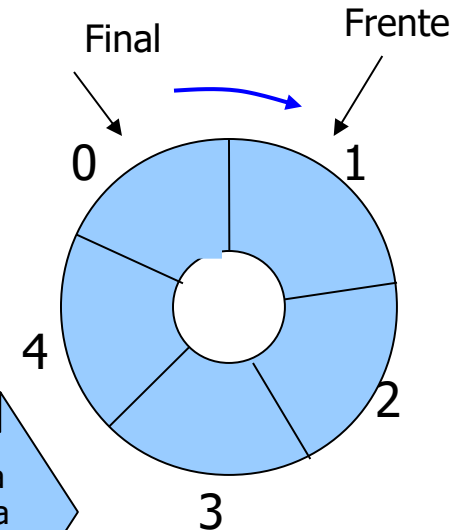
* Con todo lo visto, ¿que condición determinará.....

* **que la Cola esta llena?**

* El elemento siguiente del Final,
es el elemento del Frente



La condición para Cola
Vacía es igual que para
Cola Llena, que hacer?



* **Y que la Cola esta vacía?**

* Suponga una cola con dos elementos

* Ahora, se desencola uno

* Al remover el ultimo:

* El elemento siguiente del final es el elemento del
Frente

LSCONT: IMPLEMENTACION

```
package TipoCola;

public class ColaCircular
{
    private static final int MAXTAMQ = 99;
    protected int frente;
    protected int fin;
    protected Object [] listaCola ;
    // avanza una posición
    private int siguiente(int r)
    {
        return (r+1) % MAXTAMQ;
    }
    //inicializa la cola vacía
    public ColaCircular()
    {
        frente = 0;
        fin = MAXTAMQ-1;
        listaCola = new Object [MAXTAMQ];
    }

    public void insertar(Object elemento) throws Exc
    {
        if (!colaLlena())
        {
            fin = siguiente(fin);
            listaCola[fin] = elemento;
        }
        else
            throw new Exception("Overflow en la cola");
    }
}
```

```
public Object quitar() throws Exception
{
    if (!colaVacia())
    {
        Object tm = listaCola[frente];
        frente = siguiente(frente);
        return tm;
    }
    else
        throw new Exception("Cola vacia ");
}

public void borrarCola()
{
    frente = 0;
    fin = MAXTAMQ-1;
}

// acceso a la cola
public Object frenteCola() throws Exception
{
    if (!colaVacia())
    {
        return listaCola[frente];
    }
    else
        throw new Exception("Cola vacia ");
}

// métodos de verificación del estado de la cola
public boolean colaVacia()
{
    return frente == siguiente(fin);
}

// comprueba si está llena
public boolean colaLlena()
{
    return frente == siguiente(siguiente(fin));
}
}
```

COLA: LISTA ENLAZADA

- ✧ Con la implementación de **listas enlazadas**

- ✧ Ya no existirá para nada el problema de desperdicio de memoria

- ✧ Ni de límites máximo

- ✧ No olvidemos que ya las operaciones de la Cola

- ✧ Llamarán a las de la lista enlazada

```
public bool estaLlena(){  
    return lista.estaLlena();  
}
```

```
public bool estaVacia(){  
    return lista.estaVacia();  
}
```

```
public bool encolar(Nodo nodo){  
    return lista.insertarNodoInicio(nodo);  
}
```

```
public Object colaFinal(){  
    return lista.consultarUltimo();  
}
```

```
public Object colaFrente(){  
    return lista.consultarPrimero();  
}
```

```
public Object desencolar(){  
    return lista.sacarNodoFinal();  
}
```

Ejercicio en clase:

Números Capicúas

Un algoritmo para encontrar un número capicúa utiliza conjuntamente una *Cola* y una *Pila*.

El número se expresa en forma de una cadena de dígitos que se procesa caracter a caracter.

Cada dígito se pone en una cola y, a la vez, en una pila.

Una vez que se terminan de leer los dígitos y de ponerlos en la *Cola* y en la *Pila*, comienza el paso de comprobación: Se extraen, en paralelo, elementos de la cola y de la pila, y se comparan por igualdad.

De producirse alguna no coincidencia entre dígitos, significa que el número no es capicúa y **entonces se vacían las estructuras usadas arriba.**

```
Queue<String> q = new LinkedList();
```

Colas de Prioridad

TDA COLAS DE PRIORIDAD

* En las colas normales

- * Las operaciones están definidas en función del orden de llegada de los elementos
 - * *Al encolar un elemento ingresa al final de la cola*
 - * *Al desencolar, sale del frente de la cola*
- * En una cola, los elementos esperan por ser atendidos
 - * *Es justo, porque el que llega primero, se atiende primero*

* En una cola de prioridad

- * Prioridad
 - * *El orden de atención, no esta dado solo por el orden de llegada*
 - * *Cada elemento, tendrá asociado una cierta prioridad*
 - * *Cada elemento será “procesado”, según su prioridad*

TIPOS DE COLAS DE PRIORIDAD

- * Hay dos tipos de colas de prioridad

- * De Prioridad Ascendente

- * *EnColar: son encolados arbitrariamente(PQEnColar)*

- * *DesEnColar: se remueve el elemento mas pequeño de la cola(PQMinDesEncolar)*

- * De Prioridad Descendente

- * *EnColar: son encolados arbitrariamente*

- * *DesEnColar: se remueve el elemento mas grande de la cola(PQMaxDesEncolar)*

- * Las colas de prioridad pueden contener

- * Enteros, Reales

- * Estructuras,

- * *Estarían ordenadas en base a uno o mas campos*

DESENCOLAR EN COLAS DE PRIORIDAD

- * Al **encolar** un elemento en este tipo de cola
 - * Se encola al final de los elementos *con la misma prioridad*
- * El **desencolar** elementos de una cola
 - * Quiere decir, que ese elemento es escogido para ser “atendido”
 - * Se elige **el primer elemento** con la **mayor/menor prioridad**
 - * En las de prioridad ascendente, por ejemplo
 - * *Se busca atender primero al de menor valor en toda la cola: BUSCAR*
 - * *Y luego sacarlo*
- * Es decir, existe un conjunto de prioridades
 - * Cada prioridad tendrá un conjunto de elementos que se comportara como una cola

IMPLEMENTACION ESTATICA

- * Usando un arreglo Circular
- * Insertar los elementos ordenadamente a la cola
- * Se usa un arreglo circular
- * La inserción contempla
 - * Buscar la posición correcta para el elemento
 - * Cambiar la posición de los anteriores o posteriores
- * Al remover, no hay que buscar
 - * Se elimina el primero o el ultimo de la cola
 - * Dependiendo el orden del arreglo y
 - * El tipo de cola de prioridad

IMPLEMENTACION ESTATICA: ARREGLOS

- * Las colas de prioridad

- * Son un conjunto de colas

```
public class ColaPrioridad{  
    int numero;  
    //todo lo que defina a una prioridad  
    Cola cola;  
}
```

- * Podríamos crear un tipo ColaPrioridad

- * Que comprenda tanto el numero de la prioridad
 - * Como el resto de características de una prioridad
 - * Y la Cola

```
ColaPrioridad cp[10];
```

- * Luego tendremos un

- * Arreglo de Prioridades

```
Así, una cola de prioridad i puede acceder  
A su numero de prioridad: cp[i].numero  
A la cola en si: cp[i].cola
```

IMPLEMENTACION DINAMICA: LISTA SIMPLE

- ✧ Mediante una lista simple

- ✧ Cada nodo tendrá

- ✧ *Contenido, prioridad y enlace con el siguiente*

- ✧ La lista se mantiene ordenada por la prioridad

- ✧ Para encolar:

- ✧ *Buscar la posición correcta*

- ✧ *La lista debe permanecer ordenada*

- ✧ Para desencolar:

- ✧ *Eliminar el primer elemento*

IMPLEMENTACION DINAMICA

- * Mediante **lista de n colas**
- * Las prioridades forman una lista
- * Cada nodo de la lista tiene asociada una cola
- * Al **encolar**:
 - * Se busca la prioridad del elemento
 - * En la cola asociada se inserta al final
- * Al **desencolar**:
 - * Se busca la prioridad del elemento
 - * En la cola asociada, se remueve el primero

